Agentic Synthesis against Counterexample-Supplemented Sketches

Muness Castle Eric Rubeck

July 1, 2026

Abstract

Coding agents synthesize useful patches while losing track of intent: a prompt can describe what to build while leaving unstated which plausible implementation would be wrong. We present *agentic synthesis against counterexample-supplemented sketches*, a method for programming with agents under checkable rules. A human writes a partial program—a *sketch*—that fixes strategy and names holes; a finite corpus E stores observations and promoted counterexamples; an agent edits the sketch, deterministic code, and prompts; and a gate refuses completion until replay and semantic comparison pass every case in E.

The method imports the loop grammar of sketching and counterexample-guided inductive synthesis (CEGIS) while changing the synthesizer. In classical Sketch, a solver fills holes inside a formal language. Here, the synthesizer is an IDE-shaped coding agent whose edits span ordinary source code and LLM prompts. The guarantee is finite-corpus correctness: if the gate passes, the current artifact is correct for the promoted corpus under the replay and comparison semantics encoded by the repository. We state this guarantee explicitly, prove it from the gate definition, and show how it is useful despite being bounded.

The method came from a proprietary enterprise setting with a larger harness: a custom Cursor extension, `cursor://`-style commands, and an embedded web application through which subject-matter experts loaded production examples, corrected outputs, and helped turn those corrections into input/output specifications for the counterexample loop. The public companion gives readers a clean RosterSynth example of the same control structure.

1 Introduction

A coding agent can produce a working patch from a vague request. That is both the appeal and the risk. The agent is optimized to make progress under underspecification; the human is then left to decide whether the patch captured the rule, merely satisfied the visible test, or invented an accidental convention. The dangerous failure is a plausible implementation that closes the obvious gap while violating the policy the prompt failed to make explicit.

This paper describes a discipline for using agents as synthesizers while moving the specification out of chat history and into three visible artifacts:

1. a **sketch** $\mathcal{S}$: a partial program in prose and code-shaped structure;
2. a **counterexample-supplemented corpus** E: examples and promoted failures;
3. a **gate**: executable validation over all of E.

The agent edits ordinary repository files: the sketch, deterministic implementation code, and prompts for LLM-mediated holes. The gate is the boundary of done. When the gate finds a failure,

that failure is promoted into E, and the sketch is tightened so the next agent pass operates under a stronger contract.

The central claim is bounded and operational:

> A coding-agent workflow becomes inspectable when each counterexample is linked to the sketch clause it supplements, the implementation or prompt path that handles it, and the replay/compare check that would fail the tempting wrong patch.

This claim gives maintainers a local correctness theorem over the current corpus and a procedure for growing that corpus when the theorem is falsified.

Contributions. This paper makes four contributions.

1. It defines counterexample-supplemented sketches as a repository-native artifact for agentic coding.
2. It separates two hole-filling roles: Oracle A, deterministic code the agent maintains, and Oracle B, prompt/LLM/cassette behavior for sketch-declared narrative holes.
3. It states a finite-corpus correctness theorem for replay/compare gates and proves it from the executable gate contract.
4. It provides a sketch-level RosterSynth example and a companion repository for readers who prefer runnable evidence.

2 Originating setting

This work was motivated by a proprietary enterprise deployment. The original setting involved heterogeneous operational data, multiple downstream clients with their own rules, and subject-matter experts operating through a review interface while coding agents edited source. The deployed harness joined these pieces:

- a custom Cursor extension for invoking repository actions from the IDE;
- `cursor://`-style URL commands that connected review actions to local agent workflows;
- an embedded web application where SMEs could load production examples, inspect proposed remediations, and supply corrections;
- LLM-assisted conversion from SME corrections into input/output test specifications;
- a counterexample loop that promoted selected failures into a golden dataset; and
- non-developer operation of the loop for code and prompt implementations of a complex data-cleansing pipeline.

That origin fixes the intended problem scale. The method turns domain corrections into durable counterexamples that can constrain both deterministic code and prompt-mediated behavior.

The public repository intentionally abstracts from the enterprise deployment. It contains the paper, clean fixtures, runnable examples, tests, and provenance. Production data, proprietary extension code, client-specific rules, and private SME workflow remain outside the artifact. The evidence boundary is explicit: the public artifact demonstrates the same control structure—sketch, counterexamples, known-code anchors, dual oracles, and replay/compare gates—on a runnable example.

3 Lineage and departure from Sketch

Solar-Lezama's sketching work framed synthesis as a collaboration between programmer insight and automated search: the programmer writes a partial program, leaving holes for a synthesizer to fill [5, 6]. The programmer supplies high-level structure while the machine discharges low-level details. CEGIS adds the validation loop: generate a candidate, validate it externally, feed counterexamples back into the next synthesis step.

Our setting changes the machinery while preserving the pressure. The synthesizer becomes a coding agent inside a repository. The holes include ordinary source edits, API choices, error-shape conventions, and prompt instructions. The validator is a repo-native gate: tests, replay checks, semantic comparison, fixture runs, and source selectors.

	Sketch / CEGIS	Programming by example	This work
Human input	Partial program with holes	Input/output examples, DSL	Sketch file, examples, counterexamples, known-code anchors
Synthesizer	Solver-backed search	DSL learner / witness functions	Coding agent editing repo files
Counterexample source	Validator / solver	User examples or generated examples	Gate failure promoted into corpus E
Validation	Formal or bounded decision procedure	Consistency with examples	Replay + semantic compare + tests over E
Guarantee	Solver-relative correctness	DSL/example consistency	Finite-corpus correctness under executable gate

Programming-by-example systems such as FlashMeta and PROSE synthesize DSL programs from examples using domain-specific deduction [4, 2]. Agent benchmarks such as SWE-bench evaluate whether language models can resolve real repository issues [3]. Tests-as-prompts work studies tests as both input and evaluation for LLM code generation [1]. Agentic synthesis against counterexample-supplemented sketches combines those traditions with ordinary repository work: agents edit code and prompts under a persistent sketch, a promoted corpus, and executable gates.

4 The programming model

The method is easiest to understand as a repository contract.

Definition 1 (Sketch)**.** *A sketch $\mathcal{S}$ is a human-editable artifact that fixes the permitted strategy space and names holes. It may contain prose, tables, pseudo-code, type shapes, decision order, and explicit abstention rules. It persists in the repository and is edited when counterexamples reveal missing policy.*

Definition 2 (Corpus)**.** *The corpus $E = \{(x_i, y_i)\}_{i=1}^{n}$ is the finite set of observations the current gate must satisfy. Each input x_i is a concrete scenario or fixture. Each y_i is the expected semantic outcome, including fields replay leaves open.*

Definition 3 (Counterexample-supplemented sketch)**.** *A sketch is counterexample-supplemented when failures are promoted into E and used to revise $\mathcal{S}$. The sketch starts as an initial specification and becomes the stable surface on which the corpus teaches the next agent pass what was previously missing.*

Definition 4 (Known-code anchor)**.** *A known-code anchor is an existing source artifact that constrains the agent's implementation style: an API, result type, parser convention, error shape, fixture format, test helper, or reference implementation. It prevents the agent from satisfying the example by inventing a parallel local convention.*

The model distinguishes two implementation roles.

Oracle A: deterministic code. Oracle A is source code maintained by the agent or developer. It handles holes that are encodable in ordinary control flow: selecting a duplicate booking, choosing a work date, constructing a typed row, preserving an error convention. Oracle A should be boring: future maintainers can debug it directly.

Oracle B: narrative completion. Oracle B is the LLM-mediated path for holes the sketch marks as narrative or non-encoding: ambiguous notes, human descriptions, or policy text whose resolution should remain in the prompt layer for now. Oracle B is still constrained by the sketch and checked by the same gate. In CI, a cassette can freeze the output so the gate remains reproducible.

Hybrid resolution. A hybrid resolver runs Oracle A first. If Oracle A abstains or produces a row that replay rejects, the resolver falls back to Oracle B. This keeps deterministic code on the hot path while preserving an explicit route for narrative holes.

5 Gate semantics

The gate is the executable semantics of the corpus. It has two layers.

Definition 5 (Replay)**.** *Replay applies a candidate output r to an input state x and checks whether the state change closes the domain gap. In RosterSynth, replay computes the effective coverage delta after appends or booking modifications. A row is replay-valid when the effective delta is zero up to tolerance.*

Definition 6 (Semantic compare)**.** *Semantic compare checks fields that replay leaves open: operation kind, selected booking, work date, issue type, status mutation, or other policy-bearing columns. A row is compare-valid when it matches the golden expectation for that scenario.*

Definition 7 (Gate pass)**.** *For a strategy H and corpus E, the gate passes iff for every $(x_i, y_i) \in E$, the candidate row $r_i = H(x_i)$ is both replay-valid on x_i and compare-valid against y_i.*

Replay and compare are intentionally redundant in some cases. Replay verifies state repair. Compare rejects state-repairing rows that violate policy. The kiosk example needs both checks because appending -40 hours closes the hours math while cancellation is the policy.

6 Finite-corpus correctness

This section states the proof obligation honestly. The method proves correctness over the current promoted corpus under the repository's executable semantics. That is the right theorem for a coding-agent workflow: when a new failure appears, it becomes a new element of E, and the theorem's domain grows.

Definition 8 (E-correctness)**.** *A strategy H is E-correct with respect to replay function R and compare predicate C when, for every $(x_i, y_i) \in E$, $R(x_i, H(x_i)) = \text{true}$ and $C(y_i, H(x_i)) = \text{true}$.*

Lemma 1 (Replay soundness relative to the implementation)**.** *If the replay checker returns true for (x, r), then r satisfies the state-repair condition encoded by the replay checker for x.*

Proof. The replay checker is the executable definition of state repair. In the RosterSynth example, it evaluates appends and modifications against the observed roster state and returns true only when the effective coverage delta is closed. Therefore a true result entails exactly the state-repair condition the checker implements. This is implementation-relative soundness with the checker's encoded business-rule boundary. □

Lemma 2 (Compare soundness relative to the corpus)**.** *If semantic compare returns true for expected row y and candidate row r, then r agrees with y on every policy-bearing field implemented by the comparer.*

Proof. The comparer enumerates the checked fields and fails on mismatch. For modify rows in the worked example, it checks the operation, issue type, booking id, and status fields. Therefore a true result means no checked field differed. Again, the guarantee is relative to the comparer's encoded field set. □

Theorem 1 (Finite-corpus gate soundness)**.** *If the gate passes for strategy H on corpus E, then H is E-correct with respect to the replay and compare semantics encoded by the repository.*

Proof. By definition, the gate iterates over every $(x_i, y_i) \in E$, computes $r_i = H(x_i)$, and requires both replay and compare to pass. By the replay lemma, each accepted r_i satisfies the state-repair condition encoded by replay for x_i. By the compare lemma, each accepted r_i agrees with y_i on every field encoded by semantic compare. Therefore every corpus case satisfies both predicates, which is exactly E-correctness. □

Scope of the theorem. The theorem's scope is the promoted corpus E, the encoded replay checker, the encoded semantic comparer, and the current golden rows. The useful guarantee is narrow: the current artifact must satisfy every promoted counterexample under the executable semantics the repository exposes.

Bounded observation hypothesis. The analogy to Sketch is the bounded observation hypothesis: a small set of carefully chosen inputs can often constrain the desired implementation in practice [5]. Here that hypothesis becomes operational. The human and agent grow E when failures reveal missing policy. The corpus records the current boundary of accountability.

7 The synthesis loop

The process is a CEGIS-shaped loop with an agent in the repair role and a domain expert in the correction role.

1. Start with sketch $\mathcal{S}$, corpus E, and known-code anchors K.
2. Load a concrete example into a review surface where a domain expert can inspect the proposed output.
3. Convert the expert's correction into an input/output specification and decide whether it should be promoted into E.
4. Ask the coding agent to implement or repair Oracle A / Oracle B while preserving $\mathcal{S}$ and K.
5. Run the gate over E.
6. If all cases pass, the artifact is E-correct.
7. If a case fails, promote the failure into E, revise $\mathcal{S}$ so the missing rule is explicit, and repeat.

The important step is promotion. A failure trapped in chat is a memory leak. A failure becomes useful when it changes the corpus and the sketch so future runs carry the rule.

Why agents fit the repair role. Agents are good at local synthesis under examples and constraints. They are bad at remembering invisible constraints across sessions. Counterexample-supplemented sketches turn hidden constraints into repository artifacts. The agent can then do what it is good at: apply local edits to code and prompts under explicit validation pressure.

8 Worked example: RosterSynth kiosk double-booking

RosterSynth is the sketch-level worked example. It shows the loop through a concrete roster remediation case.

Scenario. A subject-matter expert sees a roster gap and two active bookings that explain it. The tempting repair is an adjustment that closes the arithmetic gap. The domain rule points somewhere else: when a duplicate booking explains the gap, cancel the duplicate booking selected by the sketch rule.

Counterexample. The tempting adjustment passes replay because the hours balance. It fails semantic compare because the operation is wrong. That failure becomes a counterexample in E.

Sketch update. The sketch gains the rule the failure exposed: duplicate active bookings on the pay-window end date with the same shift and hours cancel the selected duplicate when that cancellation closes the gap. Future agent passes must preserve that rule whether the implementation path is deterministic code, prompt-mediated completion, or a hybrid of both.

Gate. Replay checks that the proposed row repairs the roster state. Compare checks that the row uses the rule the sketch requires. The example demonstrates the method's central lesson: state repair and semantic repair are separate obligations.

9 Companion artifact

The companion repository exists for readers who prefer examples before abstractions. It contains the clean RosterSynth slice, generated provenance, and tests that exercise the sketch-level story above. The paper keeps implementation details out of the main argument; the repository README gives the run commands and file map.

10 Discussion

Tests name behavior; sketches name intent. Tests can verify behavior. Counterexample-supplemented sketches add the rule the failure exposed: the failing case joins E, and the sketch records what future agent passes must preserve.

Prompts implement one surface of the contract. The sketch persists as a repo artifact. Prompt text is one implementation surface alongside deterministic code, fixtures, replay, and compare.

Formal synthesis remains stronger when policy is fully encodable. Many coding-agent tasks mix encodable policy with conventions, prose, examples, and codebase-local style. This method targets that middle ground: ordinary repositories where the useful guarantee is bounded, executable, and inspectable.

Progress strengthens the constraint set. The useful metric is whether the repo now contains a stronger sketch, a sharper counterexample corpus, and checks that reject the previously tempting wrong patch.

11 Limitations

1. **Finite corpus.** The theorem covers E. New failures must be promoted.
2. **Checker quality.** Replay and compare inherit the limits of their encoded semantics.
3. **Golden quality.** A wrong golden row makes the gate enforce the wrong behavior.
4. **Sketch quality.** Hidden prose rules still require human review.
5. **Prompt drift.** Oracle B can drift when live model behavior changes; cassettes make CI reproducible while live behavior still needs review.
6. **Human judgment.** The method surfaces obligations; domain review remains part of the loop.

12 Conclusion

Agentic synthesis against counterexample-supplemented sketches treats a coding agent as a synthesizer inside a constraint environment. The human writes the sketch, curates the corpus, and promotes failures. The agent edits code and prompts. The gate supplies the finite-corpus correctness boundary. The method makes agent-written code explainable, reusable, and progressively harder to fool with the same wrong patch twice.

References

[1] Yi Cui. Tests as prompt: A test-driven-development benchmark for llm code generation, 2025.

[2] Sumit Gulwani, Oleksandr Polozov, and Rishabh Singh. Program synthesis. *Foundations and Trends in Programming Languages*, 4(1–2):1–119, 2017.

[3] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024.

[4] Oleksandr Polozov and Sumit Gulwani. Flashmeta: A framework for inductive program synthesis. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications*, OOPSLA 2015, pages 107–126. ACM, 2015.

[5] Armando Solar Lezama. *Program Synthesis by Sketching*. PhD thesis, EECS Department, University of California, Berkeley, December 2008.

[6] Armando Solar-Lezama. Program sketching. *International Journal on Software Tools for Technology Transfer*, 15(5–6):475–495, 2013.